1. Introduzione

In geologia, nel campo della modellazione ambientale, eventi distruttivi come colate detritiche, frane o colate laviche rappresentano una reale minaccia non solo per gli insediamenti urbani, ma anche per gli ecosistemi naturali.

In questi fenomeni definiamo:

- **La Colata (Flow)**: Rappresenta l'area di invasione di un evento naturale che trasporta massa (liquidi, detriti o lava).
- **L'Habitat**: Indica l'area geografica caratterizzata da particolari condizioni riproduzione o alimentazione di determinate specie di flora e fauna.

Determinare l'intersezione tra la traiettoria di una colata (F) e l'estensione di un habitat (H) è importante perché ci aiuta a quantificare l'impatto che ha avuto quell'evento distruttivo su un ecosistema. Questa intersezione viene calcolata con il rapporto tra le celle sovrapposte (ovvero quelle in cui la colata invade l'habitat) e la dimensione dell'habitat.

$$\text{Overlap}(F, H) = \frac{\sum(H_{ij} > 0 \wedge F_{ij} > 0)}{\sum(H_{ij} > 0)}$$

il risultato è un unico valore compreso tra 0 ed 1, maggiore il valore e maggiore sarà l'impatto dell'evento sull'habitat. Di fatto dobbiamo 'solamente' controllare se le celle solo attive (> 0) o meno, e quali si sovrappongono.

Il problema si può generalizzare nel caso in cui abbiamo diversi habitat e varie colate, oppure un'unica colata ma aggiornata nel tempo. In questo contesto potremmo avere N habitat ed M colate e dobbiamo calcolare una matrice risultante composta dagli Overlap di ogni coppia di matrice (H, F).

Si può ben intuire che se abbiamo vari dati riguardanti eventi catastrofici, ed i dati a disposizione sono enormi, parliamo di matrici molto estese, allora avremo bisogno di algoritmi efficienti nella gestione di dati sparsi e che sappiano sfruttare il parallelismo intrinseco di questo problema.

Noi ci concentreremo sulla creazione di un algoritmo parallelo efficiente che sappia calcolare l'Overlap Matrix dati N habitat ed M colare. Assumeremo che ogni matrice contenuta nei dati degli habitat e delle colate abbiamo la stessa dimensione.

---

2. Implementazione

Il problema, a prima vista sembra imbarazzantemente parallelo: il calcolo del singolo overlap per ogni coppia (h_i, f_i) di celle avviene cella per cella (element-wise), senza alcun bisogno di conoscere i valori vicini per produrre l'output. Tuttavia, per ricavare il valore finale di overlap serve sommare tutti i conteggi, quindi entra in gioco una riduzione che rompe questo parallelismo puro, si tratterà di un problema non imbarazzantemente parallelo.

Ho deciso di affrontare l'implementazione passo dopo passo. Prima ho testato il calcolo su due sole matrici (una H e una F), applicando come unica ottimizzazione la vettorizzazione. Poi, per gestire il caso reale con matrici multiple, ho sviluppato quattro versioni generalizzate, ognuna con i suoi pro e contro.

Adesso andremo a vedere le varie versioni che ho sviluppato.

2.1 Baseline su CPU

È la versione di partenza, interamente eseguita su CPU, che mi serve come punto di riferimento per verificare la correttezza dei dati e misurare i miglioramenti delle versioni successive.

Il vantaggio principale è la semplicità: il codice è lineare, facile da scrivere e non ha l'overhead di dover trasferire dati tra host e device. Lo svantaggio evidente sono le prestazioni, dato che, come vedremo, la CPU fatica a reggere il carico quando le matrici crescono di numero e dimensione.

2.2 GPU Naive con vettorizzazione

Qui ho fatto un primo porting elementare su GPU. Lancio il kernel $N \times M$ volte, inviando le singole matrici man mano che servono. In questa fase ho ignorato il problema della riduzione sulla scheda: il kernel fa solo il confronto vettorizzato e invia la singola matrice risultante con i conteggi, mentre la somma finale per comporre il valore di overlap la fa ancora la CPU, come anche il calcolo totale della matrice di overlap.

Data la natura element-wise di questa parte del problema, tratto le matrici come dei vettori monodimensionali. Faccio uso della vettorizzazione (impostata a 1, 2, 4, 8, 16) in modo che un singolo thread (work-item) possa confrontare più celle, da qui in avanti la vettorizzazione sarà sempre presente nelle implementazioni. In aggiunta ho compresso l'output da interi di 4 byte a `uchar` di 1 byte, dato che la nostra informazione di output è molto semplice, non serviva memorizzarla in 8 byte di dato.

In questa versione iniziamo a parallelizzare il calcolo sfruttando la GPU. D'altra parte, lo svantaggio è l'enorme collo di bottiglia dato dai trasferimenti dei dati alla GPU: fare vari trasferimenti avanti e indietro e lanciare continui kernel distrugge le prestazioni, il kernel deve comunque attendere i dati su cui dovrà lavorare, senza contare che la CPU resta comunque utilizzata per fare la riduzione a mano.

2.3 GPU con vettorizzazione e riduzione

Il salto di qualità qui sta nel delegare anche la somma alla GPU. Il kernel calcola l'overlap di ogni coppia di matrici (H, F) ed esegue direttamente una riduzione parallela per sommare sia le celle attive dell'habitat sia le sovrapposizioni. In questo modo la CPU riceve già i totali pronti e deve solo ricomporre la matrice finale.

I kernel GPU calcolano due valori, il numero di celle attive di H ed il numero di celle in overlap, ovvero attive sia in H e in F. Rimane ancora la vettorizzazione e l'uso degli `uchar` da 1 byte per i risultati lato GPU.

La riduzione avviene sfruttando la memoria locale, una memoria condivisa tra thread dello stesso work-group, utilizzando la tecnica della tree reduction. Prima ogni thread calcola il proprio valore da inserire in local memory, poi dimezzando progressivamente il calcolo, sommiamo tutti i conteggi. Una condizione a ciò è che la dimensione dei local work-group sia un sotto-multiplo esatto della dimensione della griglia di calcolo (gws).

Per chiudere l'albero della riduzione ed aggregare i parziali dei singoli work-group senza dover lanciare un secondo kernel, il primo thread di ciascun work-group (`lid == 0`) accumula il proprio totale parziale direttamente nelle variabili in memoria globale (`g_total_h` e `g_total_overlap`) tramite operazioni atomiche (`atomic_add`), quindi senza interruzioni. In questo modo l'intera riduzione viene completata con un'unica invocazione del kernel, garantendo la correttezza dei totali finali.

Adesso la GPU, una volta finito il calcolo, non invia più un'intera matrice, ma solamente una coppia di elementi, questo quindi riduce il traffico sul bus PCIe ma non in maniera significativa. Rimane ancora il grosso svantaggio riguardo il codice di lancio del kernel, resta ancora presente il ciclo lato host che costringe il kernel ad aspettare che le varie matrici vengano man mano inviate alla GPU.

2.4 GPU con griglia 2D ed invio in batch

In questa versione ho voluto risolvere il collo di bottiglia dato dai continui blocchi dovuti all'invio delle singole matrici solo quando erano necessarie. Quindi ho deciso di prepararle tutte in un unico vettore lato host e di inviarle in bath alla GPU, cercando di saturare il bus PCIe. Quindi i kernel avrebbero dovuto gestire tutte le matrici in contemporanea.

Per far si che i vari kernel gestissero tutti i confronti era necessario riorganizzare la griglia di calcolo, in questo caso sfrutteremo anche la seconda dimensione, utilizzando una griglia bidimensionale. Adesso una singola riga corrisponde esattamente ad una delle matrici ed una colonna corrisponde ai vari accoppiamenti tra (H_i, F_j).

Esempio abbiamo NxM matrici habitat e flow, ogni matrice ha dimensione c x r. L'asse x della griglia avrà dimensione pari a c x r (corrisponde ad una matrice intera). L'asse y corrisponde a

NxM, ovvero tutte le possibili coppie tra habitat e colata. Il kernel, ottenendo il valore di indice y sarà in grado di capire per quali coppie di celle stiamo facendo il calcolo. Usiamo sempre la vettorizzazione e la riduzione per fare il calcolo.

Per capire con quale coppia di matrici stiamo lavorando noi teniamo conto del numero di matrici flow, e dividiamo verticalmente la griglia in M gruppi di righe, ogni gruppo corrisponde ad una matrice H e i suoi accoppiamenti con tutti i file F.
Se immaginiamo di avere $3 \cdot 2$ matrici habitat e flow, la prima riga corrisponde all'accoppiamento (H_1, F_1) , la seconda riga ad (H_1, F_2), la terza ad (H_2, F_1).

Inoltre, essendo che a noi non interessano i valori float delle matrici ma solamente se quella specifica cella è attiva o meno, facciamo un controllo lato host che fa questo calcolo in modo da inviare ai kernel solamente dei dati che misurano 1 byte (uchar o uint) così da utilizzare ancora più banda del bus.

I risultati sono una coppia di conteggi, come nella versione precedente, che l'host (CPU) deve solamente aggiungere alla matrice risultante (overlap matrix).

I vantaggi si vedono, il kernel non deve più attendere l'invio delle varie matrici, ma può lavorare al massimo delle sue possibilità.

2.5 GPU con invio in batch asincrono

Nella precedente versione (2.4, con invio in batch), abbiamo assunto che la GPU potesse contenere tutti i dati in VRAM per eseguire il calcolo. Tuttavia, se parliamo di molte matrici e di grandi dimensioni, questo non è possibile. Quindi è necessario implementare un'ulteriore ottimizzazione che non ha bisogno di tutti i dati in VRAM, e che scambia gli input e gli output con l'host durante l'esecuzione del programma. Questa nuova versione mantiene tutte le ottimizzazioni precedenti: vettorizzazione e reduce.

La **soluzione** implica una certa sincronizzazione, la parte difficile è coordinare questi scambi di scambi di dati in modo che, mentre l'host invia i dati al device, esso sta già effettuando dei calcoli, in modo da massimizzare il throughput ed evitare i tempi morti. Quindi ho adottato una soluzione che facesse uso di **double-buffering** su GPU.

Inoltre, a differente della precedente versione (2.4), useremo il **mapping** per caricare i dati nei buffer, perché con la semplice scrittura su buffer, la GPU non può accedere direttamente agli indirizzi fisici, che il sistema potrebbe spostare o inviare su disco. Quindi il driver OpenCL deve prima copiare i dati dalla memoria host in un buffer di sistema intermedio interno alla memoria bloccata (pinned) in RAM fisica, e solo dopo i dati vengono copiati alla GPU, questa doppia copia consuma tempo di CPU e riduce il reale throughput del bus PCIe.

Usando il mapping viene allocata memoria host pinned, ovvero bloccata in RAM fisica, ottenendo un puntatore accessibile dalla GPU. Inoltre, i trasferimenti asincroni non bloccanti

sono realmente zero-overhead solo su memoria bloccata.

Questo double-buffering non è possibile implementarlo con una semplice coda OpenCL in-order. Quindi considereremo due batch di dati: Batch0 (i dati che arrivano) e Batch1 (i dati che arriveranno dopo). Noi vorremmo che mentre il kernel lavora su Batch0, la GPU carica già i dati Batch1, ed appena abbiamo i risultati di Batch0 la GPU li invia all'host, mentre Batch1 è in esecuzione. Con una coda in order tutte queste azioni devono essere fatte in maniera sequenziale.

Potremmo utilizzare anche una coda out-of-order tuttavia è preferibile usarne due perché alcuni profili driver OpenCL gestiscono meglio la concorrenza tra due code separate rispetto una coda out-of-order. Inoltre se usiamo due code in-order separate, l'ordine interno di ciascun set è garantito dalla coda stessa.

Questa soluzione a double-buffering fa uso di due code in-order e di due set distinti di buffer, ciascuno composto da un buffer per le matrici flow e da un buffer di output per i risultati. Per quanto riguarda la gestione dei volumi di dati, il limite di invio in batch viene impostato tramite gli argomenti a riga di comando, oppure calcolato automaticamente da una funzione helper (`mem_auto_conf`) che interroga i driver della GPU per verificare la memoria massima a disposizione e i vincoli delle dimensioni dei buffer.

L'implementazione si basa sull'**assunzione** che una singola matrice (ovvero un batch di dimensione 1) sia interamente memorizzabile in VRAM. Per questo motivo il codice del kernel non ha richiesto modifiche rispetto alla versione precedente (2.4), perché si è dimostrato scalabile attraverso la **riconfigurazione** a run-time delle dimensioni della griglia di calcolo.

Il fulcro della logica del programma si basa su **due cicli annidati**: quello esterno che itera sulle matrici habitat e quello interno sulle matrici flow. Nelle fasi iniziali viene caricato il blocco massimo di matrici habitat consentito; successivamente, il ciclo interno provvede a caricare i batch di matrici flow e a invocare il kernel dedicato. Il coordinamento tra le operazioni sfrutta due code di comandi (`queues`) e due istanze separate del kernel (`overlap_k`), così da prevenire race conditions. Tramite il contatore che tiene conto del numero di iterazioni del ciclo interno (`step % 2`), il codice identifica la coda e il set di buffer attivi a ogni iterazione.

A dettare il ritmo del calcolo sono le matrici habitat: a ogni iterazione del ciclo esterno lo stato viene azzerato e reimpostato. La scrittura dei nuovi dati nelle code avviene in modo sicuro sfruttando la natura sequenziale (*in-order*) delle code OpenCL, assicurando che la sovrascrittura dei buffer flow avvenga soltanto dopo che la coda ha completato le operazioni precedenti e restituito i risultati all'host.

---

3. Prestazioni

Adesso andiamo a vedere come si comportano le varie implementazioni nella risoluzione di questo problema del calcolo dell'overlap matrix. Vediamo come si comportano all'aumentare del numero di file e delle dimensioni delle matrici.

Ho utilizzato una macchina con queste componenti hardware:

- **CPU**: 11th Gen Intel(R) Core(TM) i7-1165G7 @ 2.80GHz (2.80 GHz)
- **GPU**: Intel Iris Xe Graphics G7 96EU Mobile
- **RAM**: 16 GB DDR4

3.1 Abbiamo guadagnato velocità?

La metrica che andremo a vedere per verificare se abbiamo guadagnato velocità di calcolo è lo **speed-up**. Confronta le prestazioni delle versioni GPU con quelle della versione baseline CPU.

$$\text{Speedup Kernel (Puro)} = \frac{T_{\text{CPU}}}{T_{\text{Kernel}}}$$

$$\text{Speedup End-to-End (Totale)} = \frac{T_{\text{CPU}}}{T_{\text{H2D}} + T_{\text{Kernel}} + T_{\text{D2H}} + T_{\text{CPU_post}}}$$

Lo speed-up kernel ci mostra quanto è veloce il calcolo del kernel in confronto al calcolo CPU, invece lo speed-up end-to-end ci mostra anche i rallentamenti dovuti al trasferimento dei dati e al possibile post-processing CPU.

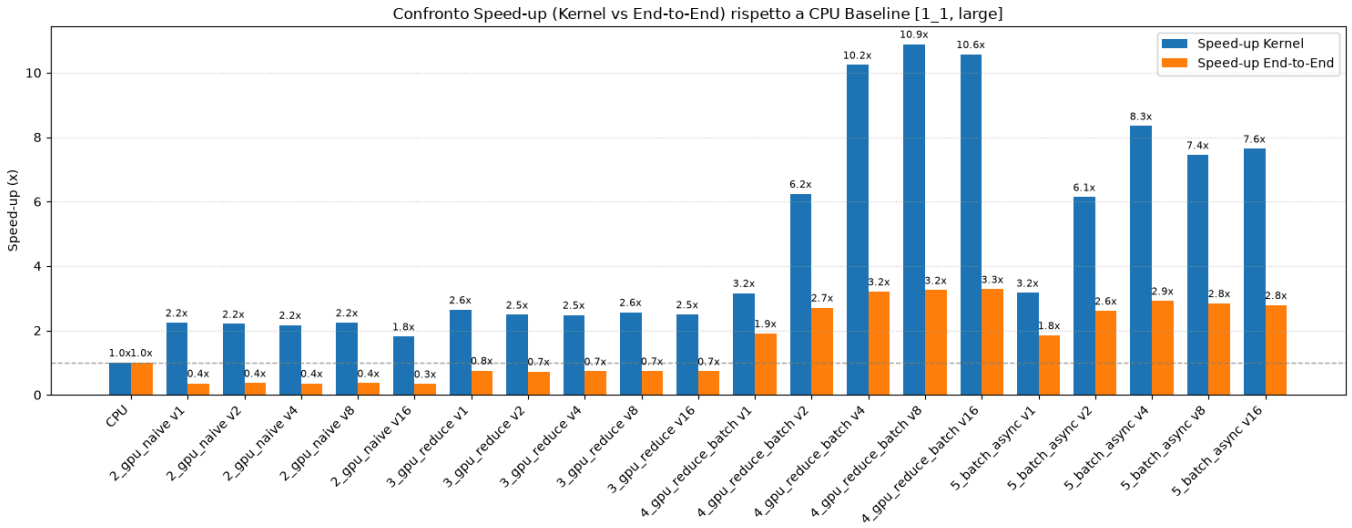


Figura 1.0: Numero di matrici: $1H$ x $1F$, Dimensioni: 4096 x 4096. Qui possiamo vedere tutte le versioni, ognuna cone le varie vettorizzazioni, a confronto.

In **Figura 1.0**, notiamo come in tutte le versioni lo speed-up kernel ci indica come effettivamente il calcolo GPU è più veloce, partendo da un moltiplicatore di 2,2x. Tuttavia nelle versioni in cui non inviamo i dati in batch, lo speed-up e2e ci indica che l'overhead dovuto al trasferimento dei file e/o al post-processing CPU inizia ad essere considerevole.

In pratica nelle versioni GPU Naive e GPU Reduce sprecavamo tantissimo tempo nelle attese di invio dei dati e, nel solo caso Naive, nel calcolo del valore della matrice finale (il post processing CPU). Possiamo vedere come la versione con Reduce primeggia nel tempo kernel, appunto perché non c'è post-processing CPU.

La versione GPU Batch si dimostra essere la migliore in assoluto, con un miglioramento delle performance che arriva ad un 10x nel kernel utilizzando vettorizzazione 4, 8, 16. Tuttavia nelle precedenti versioni, la vettorizzazione non sembra migliorare le prestazioni, ma anzi quasi peggiorano, questo perché, lavorando sulle singole coppie di matrici e dovendo comunque attender l'invio dei dati, il miglioramento non è così significativo, o forse a causa della saturazione dei registri delle varie ALU.

La versione GPU Batch asincrona sembri avere prestazione un po' peggiori rispetto alla sorella minore, questo potrebbe essere causato dalla gestione della sincronia tra le due code dei comandi in order di cui essa fa uso. Parliamo di tempi di attesa CPU, oppure gestione dei buffer come svuotarli o riempirli.

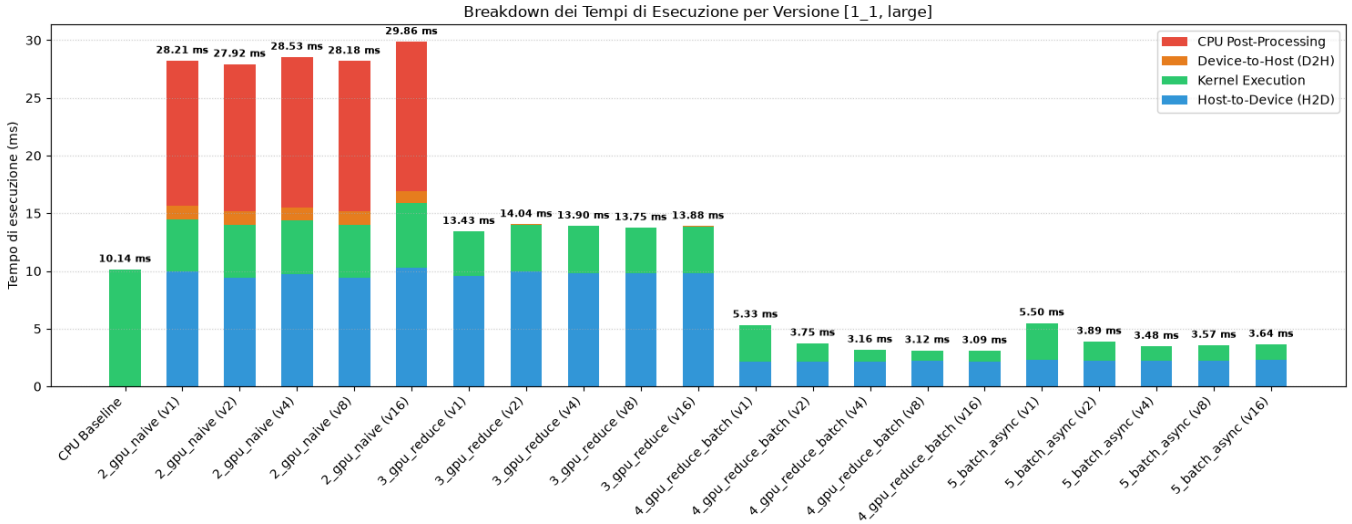


Figura 1.1: Qui possiamo vedere un breakdown dei tempi d calcolo.

Come possiamo vedere in **Figura 1.1**, le versioni iniziali, parlo di quelle naive e quella reduce soffrono molto dell'overhead causato dai blocchi della CPU.

Nelle versioni con batch molto del tempo è investito nell'invio dei dati al device, inoltre nelle versioni asincrone, il kernel impiega più tempo, proprio perchè deve attende quei possibili blocchi dettati dalla coordinazione delle due code

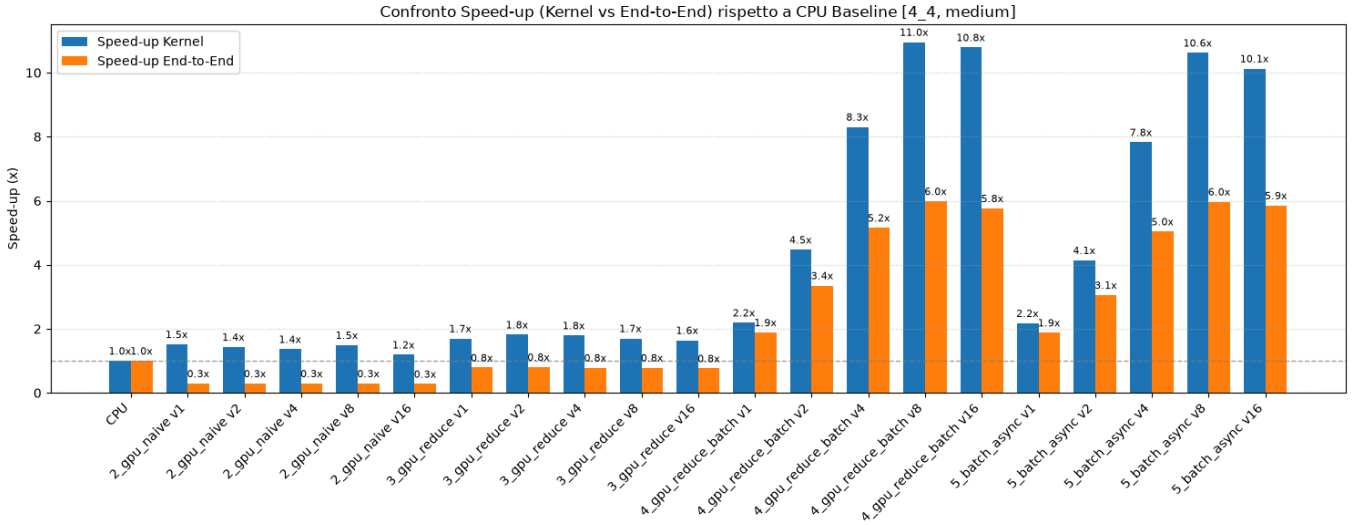


Figura 2.0: Numero di matrici: $4H$ x $4F$, Dimensioni: 2048 x 2048

In **Figura 2.0** vediamo che all'aumentare del numero di file vediamo la vera potenza delle versioni che fa uso di trasferimento in Batch, sono senza dubbio quelle con minore overhead e con le prestazioni migliori, il kernel 6 volte più veloce. Tuttavia vediamo come è aumentato di molto l'overhead nelle versioni precedenti, il miglioramento è quasi poco significativo.

Adesso che sono aumentati il numero di matrici, vediamo come le prestazioni della versione batch asincrona siano pressoché identiche alla versione precedente senza asincronia. Indicando come la gestione delle due code dei comandi funzioni.

In queste due versioni in batch vediamo anche come all'aumentare della vettorizzazione aumentino anche le prestazioni. Questo grazie all'implementazione del kernel che riduce di molto le dimensioni della griglia di calcolo all'aumentare di questo valore di vettorizzazione.

La versione batch asincrona ad una sola vettorizzazione soffre molto l'overhead causato dalla gestione della sincronia.

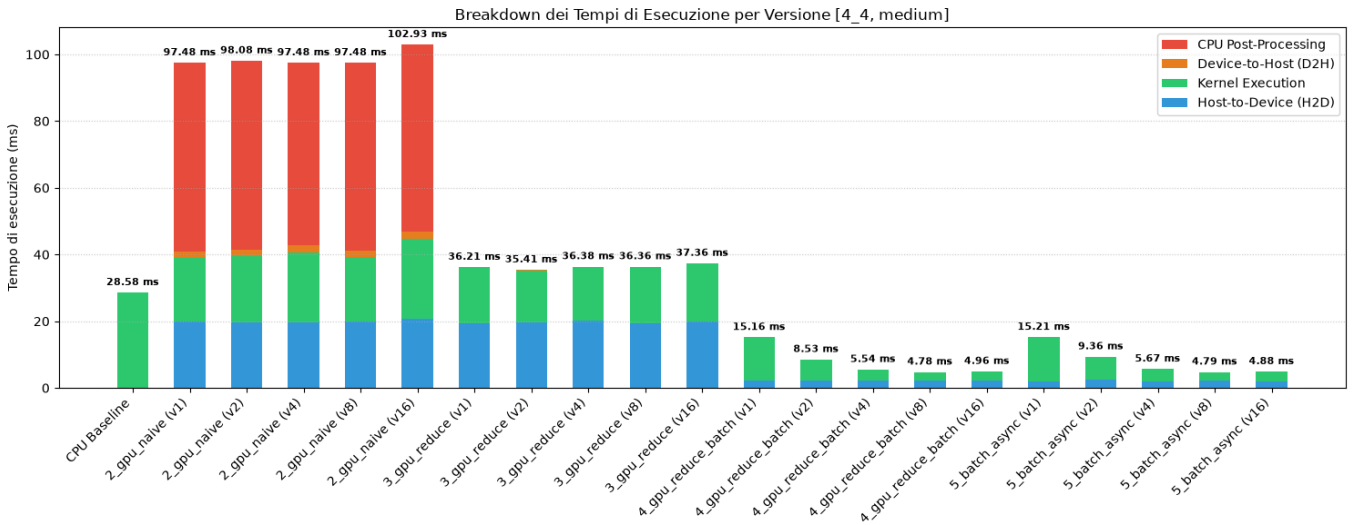


Figura 2.1: Ecco il break down dei tempi del grafico appena visto.

Infatti vediamo come la versione batch asincrona con vettorizzazione 1 soffri molto del problema appena descritto.

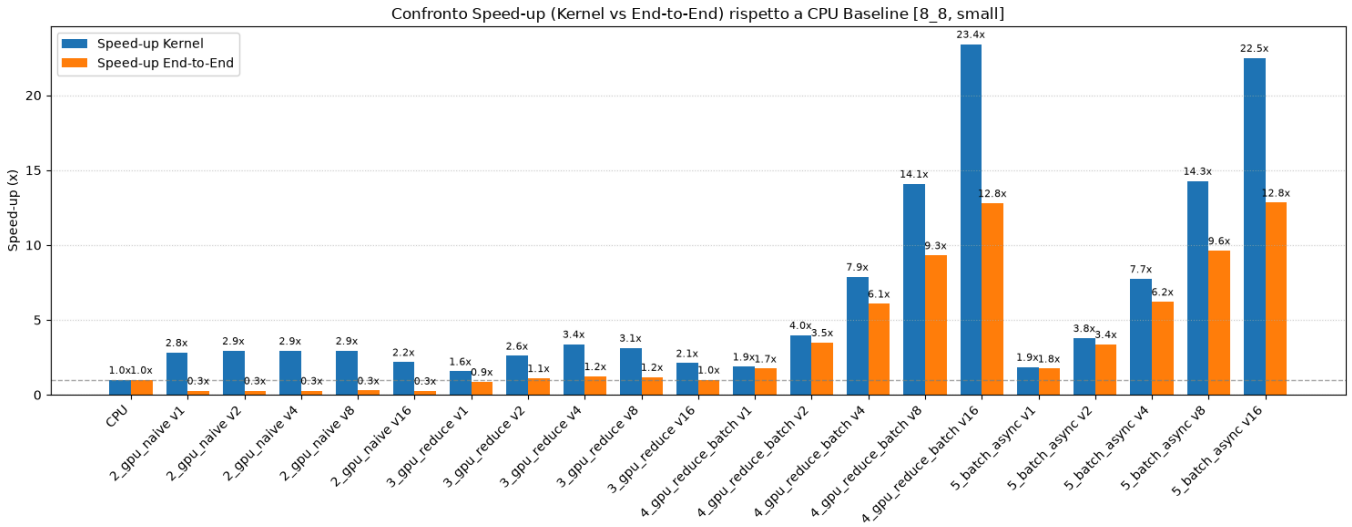


Figura 3.0: Numero di matrici: $8H$ x $8F$, Dimensioni: 1024 x 1024

Vediamo il caso in cui abbiamo molte matrici ma di dimensioni non enormi. La versione in batch è tanto più veloce, l'ottimizzazione sui trasferimenti in batch ha portato ad uno speed-up totale del 23x nella versione con vettorizzazione 16. E con queste dimensioni delle matrici iniziano ad esprimere un po di potenza anche la altre versioni GPU, forse soffrono l'overhead dovuto al post processing CPU, perché quello aumenta all'aumentare della dimensione dei dati.

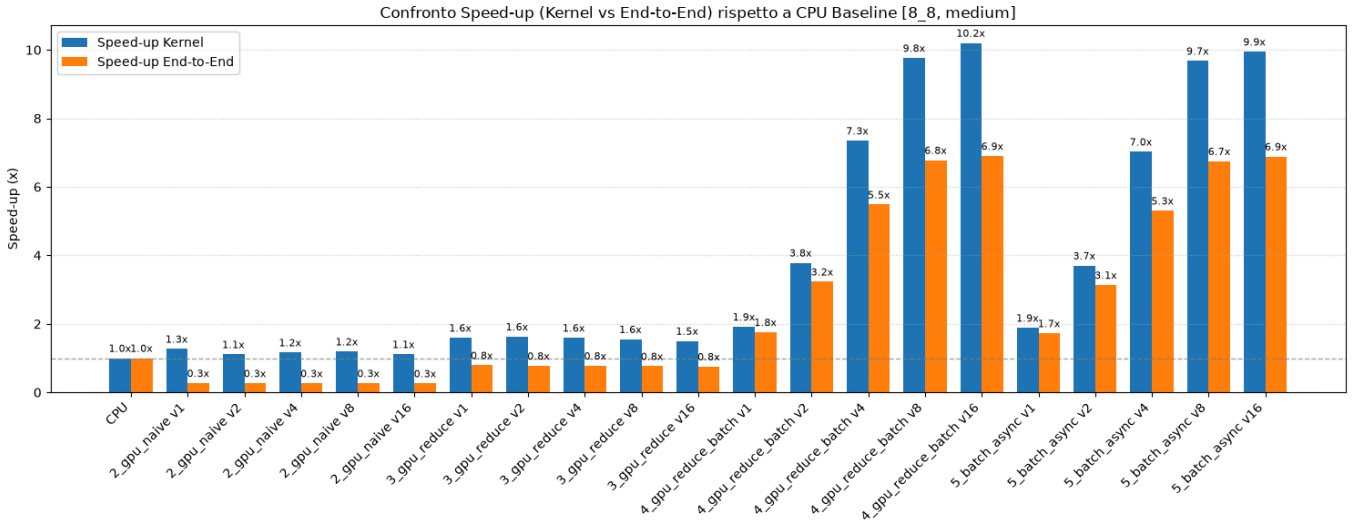


Figura 3.1: Numero di matrici: $8H$ x $8F$, Dimensioni: 2048 x 2048

Se in Figura 3.0, avevamo un monopolio della vettorizzazione 16 per entrambe le versioni batch (asincrona e non), adesso che abbiamo aumentato le dimensioni delle matrici, vediamo come anche altre vettorizzazione iniziano ad essere pi§ efficienti, perché subiscono meno l'overhead causato dai vari invii.

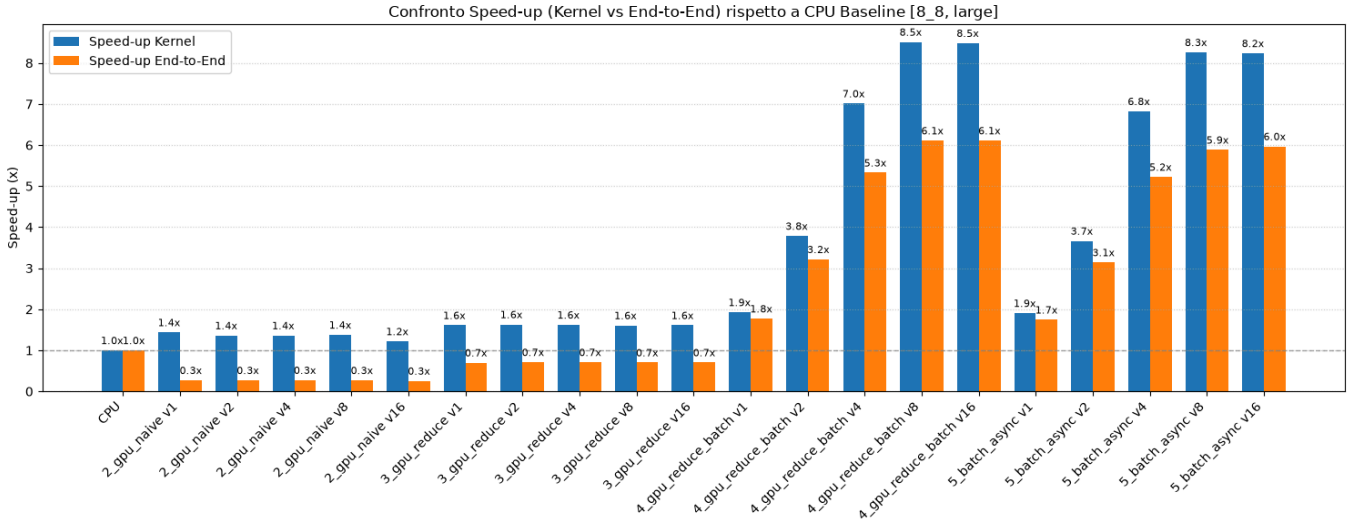


Figura 3.2: Numero di matrici: $8H$ x $8F$, Dimensioni: 4096 x 4096

In Figura 3.2 vediamo il test massimo che ho fatto, 64 accoppiamenti tra matrici di grandi dimensioni. Vediamo come le versioni batch continuano ad essere le migliori, e la vettorizzazione funziona alla grande, 8 e 16 sono pressoché identiche per entrambe le versioni.

3.2 Utilizzo della banda del bus PCIe

Per vedere come le varie versioni utilizzassero la memoria ho usato la metrica dell'effective bandwidth, che mette in relazione i byte scambiati con l'host e il tempo di esecuzione del kernel.

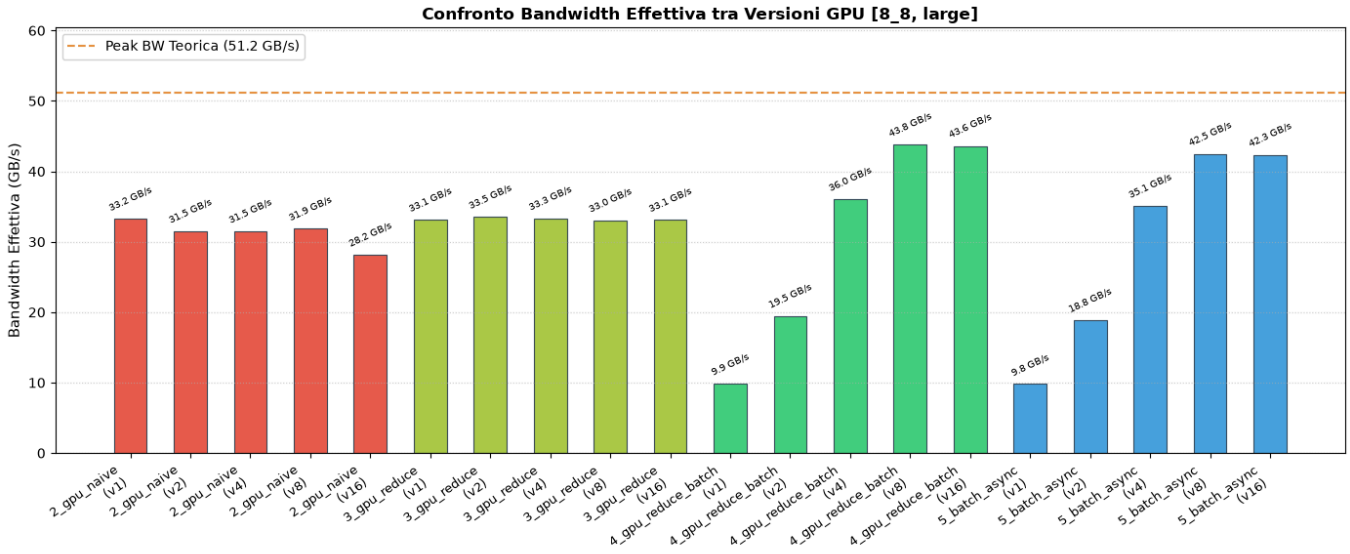


Figura 4.0 :Numero di matrici: $8H$ x $8F$, Dimensioni: 4096 x 4096

Notiamo come la EB (effective bandwidth) della versione naive sia buona, molto simile a quella della versione con reduce. Tuttavia noto qualcosa di strano nei valori per le versioni batch, a versione con vettori ad 1 sfrutta molto poco la memoria, forse perché usiamo pochi elementi per thread, mentre all'aumentare della vettorizzazione aumenta anche l'uso della banda. Le

versioni che usa la banda al meglio sono quelle batch con vettorizzazione 8 e 16. Dimostrando di ancora una volta che l'implementazione dell'asincronia funziona molto bene.

Se avessi calcolato questa metrica usando il tempo totale e non il tempo del kernel, avremmo avuto valori che avrebbero riflesso ancora più al meglio l'uso della memoria.

---

4. Conclusione e Sviluppi Futuri

In questo progetto abbiamo affrontato il calcolo dell'Overlap Matrix tra habitat e colate, partendo da una semplice baseline su CPU fino ad arrivare a versioni GPU

Con questo progetto ho voluto esplorare come affrontare un problema che, pur partendo da una base element-wise, nasconde difficoltà dovute alla riduzione e alla gestione della memoria. Il passaggio dalla versione baseline CPU fino all'implementazione batch su griglia 2D mostra chiaramente dove si trovino i veri limiti prestazionali nelle applicazioni reali su GPU.

I risultati che abbiamo ottenuto confermano che parallelizzare solo la logica di calcolo lasciando indietro la riduzione (come nella versione Naive) sposta solo il collo di bottiglia sulla CPU, vanificando quasi del tutto il vantaggio ottenuto. La gestione dei trasferimenti host-device domina l'intero tempo di esecuzione. Solo organizzando il problema per inviare i dati in blocco (batch) e mappando la griglia di lavoro su due dimensioni si riesce a saturare il bus PCIe e ad abbattere i tempi morti, raggiungendo speed-up end-to-end notevoli (fino a 23x nei test con molte matrici).

La vettorizzazione offre una marcia in più quando associata alla saturazione della banda, mentre rischia di essere poco utile o controproducente se il kernel trascorre la maggior parte del tempo ad attendere i dati.

Siamo anche riusciti ad implementare una versione ottimale in maniera safe, la gestione dell'asincronia nella versione batch rende quella soluzione scalabile su quasi ogni dispositivo.

Sebbene l'uso di tipi a 1 byte (`uchar`) abbia già ridotto significativamente l'uso della memoria rispetto ai float o interi a 4 byte originari, c'è ancora un margine di miglioramento estremo per quanto riguarda il consumo della banda.

Dato che l'unica informazione rilevante per ogni cella è binaria (attiva o non attiva), un singolo byte memorizza un'informazione che richiederebbe formalmente un solo bit.

Una naturale evoluzione dell'algoritmo consiste nel lavorare direttamente sulle coppie di bit. Lato host possiamo raggruppare lo stato di 8 celle in un unico byte. I kernel GPU potrebbero calcolare i totali tramite primitive veloci hardware come il **popcount** (conteggio dei bit a 1).

Adottare questo approccio permetterebbe di dividere ulteriormente per un fattore compreso tra 4 e 8 il traffico di dati sul bus PCIe, riducendo al minimo sia l'overhead di trasferimento Host-to-Device sia il consumo di memoria locale nei work-group durante la tree reduction.